

FINAL PROJECT DUE July 2, 2026 by Noon EST

1 Directions

Given the protocol specification and scenario below, model the protocol in CPSA and state if the protocol fits the use case in the scenario and why. Then, state if either you believe the protocol satisfies the lab's security goals, or the protocol includes issues/attacks which compromise the goals.

2 Deliverables

1. A functioning CPSA model of the protocol described in the spec below. Your model should be able to be compiled and ran on any system. The shapes it produces should be the same shapes used in the following deliverables.
2. Prepare a brief 5-10 minute presentation that includes a brief description of the protocol, screenshots of the shapes your CPSA model produced, and a high-level description of your analysis. Be prepared to present this on the last day of the workshop!
3. You should also prepare a writeup, approximately one page in length, that describes the protocol, whether the protocol is a good fit for the scenario, and its adherence to security goals in the scenario. You should include some pictures of your shapes in the writeup (this will help you fill the page). You should also include a works cited for any materials that you reference. This may include the workshop slides, the CPSA manual, and any other CPSA code that you inspect. You should make an attempt to decide which assumptions make sense in this scenario and clearly state your reason for why in your write up.

3 Definition 1: TLS

We define traces that establish a TLS handshake between a client C and a server S . Let the following be terms within the trace:

$$\begin{aligned} C, S, CA &\in Name \\ r_C, r_S &\in Nonce \\ e_S, e_C &\in Expt \end{aligned}$$

Let $hash$ be an ideal hash function, enc be an ideal encryption function with secret key for a user U be called $sk(U)$ and public key $pk(U)$. Let the operator $\parallel$ represent concatenation between two variables.

$$\begin{aligned} Cert_S &= S \parallel pk(S) \parallel enc(hash(S, g^{e_S}), sk(CA)) \\ S_{KE} &= g^{e_S} \parallel enc(hash(r_C \parallel r_S \parallel g^{e_S}), sk(S)) \\ PMS &= g^{e_S \times e_C} \\ C_{WRITE} &= hash(PMS, r_C, r_S, \text{"client-write"}) \\ S_{WRITE} &= hash(PMS, r_C, r_S, \text{"server-write"}) \\ C_{FMESG} &= r_C \parallel r_S \parallel Cert_S \parallel S_{KE} \\ C_{FIN} &= enc(hash(PMS \parallel \text{"client-fin"} \parallel C_{FMESG}), C_{WRITE}) \\ S_{FIN} &= enc(hash(PMS \parallel \text{"server-fin"} \parallel C_{FMESG} \parallel C_{FIN}), S_{WRITE}) \end{aligned}$$

Let trace $Tr_{TLS-C}(C, S, CA, r_C, r_S, e_S, e_C)$:

1. $+ r_C$
2. $- r_S \parallel Cert_S \parallel S_{KE}$
3. $+ g^{e_C} \parallel C_{FIN}$
4. $- S_{FIN}$

Let the complementary trace $Tr_{TLS-S}(C, S, CA, r_C, r_S, e_S, e_C)$:

1. $- r_C$
2. $+ r_S \parallel Cert_S \parallel S_{KE}$
3. $- g^{e_C} \parallel C_{FIN}$
4. $+ S_{FIN}$

4 Definition 2: SCEP Token

Let

$$Tok_{X,Y} = \text{"Token"} \parallel X \parallel pk(X) \parallel Y \parallel pk(Y) \parallel data$$

be the token. We then define the actual SCEP token as:

$$T_{X,Y} = Tok_{X,Y} \parallel enc(hash(Tok_{X,Y}), sk(Y))$$

5 Definition 3: Server Connection Establishment Protocol (SCEP)

We define traces that carry out SCEP between $\mathcal{S}$ and a responding infrastructure server K or H , which we define as $\mathcal{W}$. As before, we first define the terms that we use within the traces:

$$\begin{aligned}\mathcal{S}, \mathcal{W}, M, CA &\in \textit{Name} \\ r_{\mathcal{S}}, r_{\mathcal{W}}, r'_{\mathcal{S}}, r'_{\mathcal{W}}, \omega &\in \textit{Nonce} \\ e_{\mathcal{S}}, e_{\mathcal{W}} &\in \textit{Expt}\end{aligned}$$

$T_{\mathcal{S},M}$ and $T_{\mathcal{W},CA}$ are as in Definition 2.

$$\begin{aligned}\mathcal{S}_{WRITE} &= \textit{hash}(g^{e_{\mathcal{S}} \times e_{\mathcal{W}}}, r'_{\mathcal{S}}, r'_{\mathcal{W}}, \text{“client-write”}) \\ \mathcal{W}_{WRITE} &= \textit{hash}(g^{e_{\mathcal{S}} \times e_{\mathcal{W}}}, r'_{\mathcal{S}}, r'_{\mathcal{W}}, \text{“server-write”})\end{aligned}$$

Let $Tr_{SCEP-SW}(\mathcal{S}, \mathcal{W}, M, CA, r_{\mathcal{S}}, r_{\mathcal{W}}, r'_{\mathcal{S}}, r'_{\mathcal{W}}, \omega, e_{\mathcal{S}}, e_{\mathcal{W}})$:

1. $Tr_{TLS-C}(\mathcal{S}, \mathcal{W}, CA, r'_{\mathcal{S}}, r'_{\mathcal{W}}, e_{\mathcal{S}}, e_{\mathcal{W}})$ (Definition 1)
2. $+ \textit{enc}(r_{\mathcal{S}} \parallel T_{\mathcal{S},M}, \mathcal{S}_{WRITE})$
3. $- \textit{enc}(\omega \parallel r_{\mathcal{W}} \parallel T_{\mathcal{W},CA} \parallel \textit{enc}(\textit{hash}(\text{“server-mutauth”}, r_{\mathcal{S}}, r_{\mathcal{W}}, T_{\mathcal{W},CA}), pk(\mathcal{W})), \mathcal{W}_{WRITE})$
4. $+ \textit{enc}(\omega \parallel \textit{enc}(\textit{hash}(\text{“client-mutauth”}, r_{\mathcal{S}}, r_{\mathcal{W}}, T_{\mathcal{S},M}), pk(\mathcal{S})), \mathcal{S}_{WRITE})$

Let $Tr_{SCEP-WS}$ be complementary to $Tr_{SCEP-SW}$, such that $Tr_{SCEP-WS}$ (1) includes Tr_{TLS-S} rather than Tr_{TLS-C} , and (2) inverts the directions of terms 2, 3, and 4.

6 Protocol

To negotiate the protocol, the Client C and the Server S will authenticate using TLS (Definition 3). Then they will negotiate SCEP (Definition 5) using the SCEP token (Definition 4).